```
anirudh@LAPTOP-H9V5A2PN:~/OSLAB$ gcc m39.c -o m39
anirudh@LAPTOP-H9V5A2PN:~/OSLAB$ ./m39
==========================================
 SIGNAL DELIVERY AND PROCESS GROUP ANALYSIS
==========================================

Parent PID       : 1868
Parent PGID      : 1868
Parent Session ID: 377

Blocked Signals:

--- CHILD PROCESS ---
Child PID        : 1869
Child PPID       : 1868
Child PGID       : 1868
Child Session ID : 377

Child waiting for signals...

--- PARENT PROCESS ---
Sending SIGTERM to child...

Received signal: 15
SIGTERM received
^C
Received signal: 2
SIGINT (Ctrl+C) received

Received signal: 2
SIGINT (Ctrl+C) received
^C
Received signal: 2
SIGINT (Ctrl+C) received

Received signal: 2
SIGINT (Ctrl+C) received
exit
^C
Received signal: 2
SIGINT (Ctrl+C) received

Received signal: 2
SIGINT (Ctrl+C) received
^C
Received signal: 2

Received signal: 2
SIGINT (Ctrl+C) received
SIGINT (Ctrl+C) received
^C
Received signal: 2
SIGINT (Ctrl+C) received

Received signal: 2
SIGINT (Ctrl+C) received
^C
Received signal: 2
SIGINT (Ctrl+C) received

Received signal: 2
SIGINT (Ctrl+C) received
```

```c
/* signal_analysis.c
 *
 * Analyze Signal Delivery, Signal Masks, Process Groups,
 * Terminal Signals and Signal Behavior.
 *
 * Compile:
 * gcc signal_analysis.c -o signal_analysis
 */

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <sys/types.h>
#include <sys/wait.h>

void signal_handler(int sig)
{
    printf("\nReceived signal: %d\n", sig);

    if (sig == SIGINT)
        printf("SIGINT (Ctrl+C) received\n");
    else if (sig == SIGTSTP)
        printf("SIGTSTP (Ctrl+Z) received\n");
    else if (sig == SIGCONT)
        printf("SIGCONT received - process resumed\n");
    else if (sig == SIGTERM)
        printf("SIGTERM received\n");
}

void print_signal_mask()
{
    sigset_t mask;

    sigprocmask(SIG_BLOCK, NULL, &mask);

    printf("\nBlocked Signals:\n");

    for (int i = 1; i < NSIG; i++)
    {
        if (sigismember(&mask, i))
            printf("Signal %d is blocked\n", i);
    }
}
```

```c
int main()
{
    pid_t pid;

    signal(SIGINT, signal_handler);
    signal(SIGTSTP, signal_handler);
    signal(SIGCONT, signal_handler);
    signal(SIGTERM, signal_handler);

    printf("================================================\n");
    printf(" SIGNAL DELIVERY AND PROCESS GROUP ANALYSIS\n");
    printf("================================================\n");

    printf("\nParent PID       : %d\n", getpid());
    printf("Parent PGID      : %d\n", getpgrp());
    printf("Parent Session ID: %d\n", getsid(0));

    print_signal_mask();

    pid = fork();

    if (pid < 0)
    {
        perror("fork");
        exit(EXIT_FAILURE);
    }

    if (pid == 0)
    {
        printf("\n--- CHILD PROCESS ---\n");

        printf("Child PID        : %d\n", getpid());
        printf("Child PPID       : %d\n", getppid());
        printf("Child PGID       : %d\n", getpgrp());
        printf("Child Session ID : %d\n", getsid(0));

        printf("\nChild waiting for signals...\n");

        while (1)
        {
            pause();
        }
    }
    else
    {
```

```
else
{
    sleep(2);

    printf("\n--- PARENT PROCESS ---\n");
    printf("Sending SIGTERM to child...\n");

    kill(pid, SIGTERM);

    waitpid(pid, NULL, 0);

    printf("Child terminated.\n");

    printf("\nSignal behavior analysis completed.\n");
}

return 0;
```

```c
/* resume_stopped_jobs.c
 *
 * Resume Stopped Jobs, Send Continue Signals,
 * Update Job Status, Maintain Job Table,
 * Verify Execution and Test Background Recovery.
 *
 * Compile:
 * gcc resume_stopped_jobs.c -o resume_stopped_jobs
 */

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <sys/types.h>
#include <sys/wait.h>

#define MAX_JOBS 20

typedef enum
{
    RUNNING,
    STOPPED,
    DONE
} JobState;

typedef struct
{
    int job_id;
    pid_t pid;
    pid_t pgid;
    JobState state;
    char command[100];
} Job;

Job jobs[MAX_JOBS];
int job_count = 0;

const char *state_name(JobState state)
{
    switch (state)
    {
        case RUNNING:
            return "Running";

        case STOPPED:
            return "Stopped";

        case DONE:
            return "Done";

        default:
            return "Unknown";
    }
}

void add_job(pid_t pid, pid_t pgid, const char *command)
{
    if (job_count >= MAX_JOBS)
    {
        printf("Job table is full.\n");
        return;
    }

    jobs[job_count].job_id = job_count + 1;
    jobs[job_count].pid = pid;
    jobs[job_count].pgid = pgid;
    jobs[job_count].state = RUNNING;

    snprintf(
        jobs[job_count].command,
        sizeof(jobs[job_count].command),
        "%s",
        command
    );

    job_count++;
}

void update_job_status()
{
    int status;

    for (int i = 0; i < job_count; i++)
    {
        if (jobs[i].state == DONE)
            continue;
```

```
    for (int i = 0; i < job_count; i++)
    {
        if (jobs[i].state == DONE)
            continue;

        pid_t result = waitpid(
            jobs[i].pid,
            &status,
            WNOHANG | WUNTRACED | WCONTINUED
        );

        if (result == jobs[i].pid)
        {
            if (WIFSTOPPED(status))
            {
                jobs[i].state = STOPPED;

                printf(
                    "Job [%d] stopped by signal %d\n",
                    jobs[i].job_id,
                    WSTOPSIG(status)
                );
            }
            else if (WIFCONTINUED(status))
            {
                jobs[i].state = RUNNING;

                printf(
                    "Job [%d] continued\n",
                    jobs[i].job_id
                );
            }
            else if (WIFEXITED(status) || WIFSIGNALED(status))
            {
                jobs[i].state = DONE;

                printf(
                    "Job [%d] completed\n",
                    jobs[i].job_id
                );
            }
        }
    }
}

void list_jobs()
{
    update_job_status();

    printf("\n==========================================\n");
    printf("                  JOB TABLE\n");
    printf("==========================================\n");

    if (job_count == 0)
    {
        printf("No jobs available.\n");
        return;
    }

    printf(
        "%-8s %-10s %-10s %-12s %s\n",
        "Job ID",
        "PID",
        "PGID",
        "Status",
        "Command"
    );

    printf(
        "------------------------------------------------------------\n"
    );

    for (int i = 0; i < job_count; i++)
    {
        printf(
            "%-8d %-10d %-10d %-12s %s\n",
            jobs[i].job_id,
            jobs[i].pid,
            jobs[i].pgid,
            state_name(jobs[i].state),
            jobs[i].command
        );
    }
}

int find_job(int job_id)
{
    for (int i = 0; i < job_count; i++)
    {
```

```c
void start_background_job()
{
    pid_t pid = fork();

    if (pid < 0)
    {
        perror("fork");
        return;
    }

    if (pid == 0)
    {
        setpgid(0, 0);

        printf(
            "\nBackground job started.\n"
            "PID: %d\n"
            "PGID: %d\n",
            getpid(),
            getpgrp()
        );

        for (int i = 1; i <= 15; i++)
        {
            printf(
                "Job PID %d running... %d/15\n",
                getpid(),
                i
            );

            sleep(1);
        }

        printf("Background job completed.\n");

        exit(0);
    }

    setpgid(pid, pid);

    add_job(pid, pid, "background-task");

    printf(
        "Started Job [%d] with PID %d\n",
        job_count,
        pid
    );
}

int main()
{
    int choice;
    int job_id;

    printf("============================================\n");
    printf("       JOB CONTROL AND BACKGROUND RECOVERY\n");
    printf("============================================\n");

    while (1)
    {
        printf("\n");
        printf("1. Start Background Job\n");
        printf("2. List Jobs\n");
        printf("3. Stop Job\n");
        printf("4. Resume Stopped Job\n");
        printf("5. Update Job Status\n");
        printf("6. Exit\n");

        printf("\nEnter choice: ");
        scanf("%d", &choice);

        switch (choice)
        {
            case 1:
                start_background_job();
                break;

            case 2:
                list_jobs();
                break;

            case 3:
                printf("Enter Job ID: ");
                scanf("%d", &job_id);

                stop_job(job_id);
                break;
```

```
            case 2:
                list_jobs();
                break;

            case 3:
                printf("Enter Job ID: ");
                scanf("%d", &job_id);

                stop_job(job_id);
                break;

            case 4:
                printf("Enter Job ID to resume: ");
                scanf("%d", &job_id);

                resume_job(job_id);
                break;

            case 5:
                update_job_status();
                list_jobs();
                break;

            case 6:
                printf("\nExiting...\n");

                for (int i = 0; i < job_count; i++)
                {
                    if (jobs[i].state == RUNNING ||
                        jobs[i].state == STOPPED)
                    {
                        kill(-jobs[i].pgid, SIGTERM);
                    }
                }

                return 0;

            default:
                printf("Invalid choice.\n");
        }
    }

    return 0;
}
```

```
    return 0;
}
anirudh@LAPTOP-H9V5A2PN:~/OSLAB$ ./m40
==========================================
       JOB CONTROL AND BACKGROUND RECOVERY
==========================================

1. Start Background Job
2. List Jobs
3. Stop Job
4. Resume Stopped Job
5. Update Job Status
6. Exit

Enter choice:
```